TORSHIELD: A Transparent In-Kernel Authenticated Tunnel with Fail-Closed TC Steering

Roshan Srikanth
Amrita Vishwa Vidyapeetham
Email: roshansrikanth21@gmail.com

Dr. S. UdhayaKumar
Amrita Vishwa Vidyapeetham
Email: sudhayakumar@ch.amrita.edu

***Abstract*—Application-level encryption protects payloads but leaves metadata visible and requires per-application changes. TORSHIELD is a Linux kernel module that transparently encrypts peer-destined IPv4/IPv6 traffic in UDP using ChaCha20–Poly1305, a 16-byte authenticated header, per-peer 64-bit sequence numbers, a 1024-packet replay window, and 64-byte padding. The same wire format is used by Netfilter, a route-based L3 device, and TC clsact/eBPF steering. For TC, a kernel guard at `LOCAL_OUT/FORWARD` drops peer-destined packets when the egress classifier or qdisc is removed. We report captured results: 1 MB IPv4 and 256 KB IPv6 TCP transfers were byte-exact; independent AEAD verification accepted 1054/1054 tunnel frames; deletion tests blocked 8/8 local sends and 8/8 forwarded packets with zero outer packets; a 30 s four-stream native VM run sustained 3.40 Gbit/s with flat unreclaimable memory. We also report four defects found and fixed. TORSHIELD is not Tor, provides no anonymity, and does not replace WireGuard or IPsec.**

***Index Terms*—Linux kernel, network security, authenticated encryption, ChaCha20–Poly1305, Netfilter, eBPF, traffic control, replay protection.**

I. INTRODUCTION

Network operators increasingly classify, throttle, and block traffic using addresses, ports, flow statistics, packet sizes, and timing. Transport Layer Security (TLS) protects content, but it does not hide the metadata that surrounds a connection, and it requires each application to use it. A kernel-level mechanism can apply a uniform policy to every socket without changing applications, which is useful for legacy software, testbeds, and controlled deployments.

This paper describes the current TORSHIELD design. An earlier draft of this work described a single-byte XOR transform. We explicitly withdraw those claims. A single-byte XOR is trivially recoverable from known plaintext, is length-preserving, and provides no integrity; rotating the byte does not repair those properties. The XOR prototype is retained only as a historical baseline. The system evaluated here is a different datapath: an authenticated UDP tunnel with per-peer keys and a replay window.

TORSHIELD is deliberately narrow. It is *not* Tor and provides no anonymity, onion routing, or censorship circumvention. It does not disguise protocol signatures, shape timing, or hide outer addresses and ports. It is also not a replacement for WireGuard or IPsec: it has no handshake and no forward secrecy, and its control plane is a set of procfs files. The contribution is an engineering and measurement study of transparent in-kernel tunneling and of a fail-closed TC steering path that is difficult to obtain when Netfilter and TC execute at different points in the stack.

A. Contributions

- A kernel datapath and wire format for transparent peer-destined IPv4/IPv6 traffic: ChaCha20–Poly1305, a 32-bit key identifier, 64-bit per-peer sequence numbers, 64-byte padding, a 1024-packet replay window, and per-peer allowed prefixes with authenticated roaming.
- One wire format shared by Netfilter, a route-based L3 device, and TC clsact/eBPF steering, plus a kernel-side fail-closed TC guard at `LOCAL_OUT`/`FORWARD` and loader map completeness. We test classifier and qdisc deletion on local and forwarded paths.
- An empirical study on two kernel versions and a native VM, with independent AEAD verification, deletion tests, manual rekey, a rate limiter, and a 30 s sustained run. We report four defects found and fixed, and we withdraw the draft's unverified CPU, latency, footprint, stability, and distribution claims.

II. THREAT MODEL AND SCOPE

a) Assets: The confidential payload is the inner IP packet: its addresses, transport headers, and application data. The outer IP/UDP headers (including TTL and DSCP), the `TS` magic, the key identifier, the sequence number, ciphertext length, direction, timing, and approximate size bucket remain visible.

b) Adversary: We consider (i) a passive on-path observer that records and classifies packets; (ii) an active off-path sender that injects, replays, or mutates outer datagrams; and (iii) an unprivileged local process that can create sockets but cannot read kernel memory or configure the module. We do not defend against a compromised endpoint, a root attacker who can read keys or modify the kernel, or an adversary who can observe both endpoints.

c) Assumptions: The kernel and the module are trusted. Each peer has a 256-bit key configured out of band. Root configures the peer table, mode, and (for TC) the loader. The classifier is attached before the mode is switched to `tc`.

d) Non-goals: TORSHIELD provides confidentiality, integrity, and replay resistance for tunneled packets. It does not provide anonymity, traffic-analysis resistance, protocol

obfuscation, forward secrecy, automatic key agreement, per-network-namespace policy, or a general denial-of-service defense. Stream mode is legacy, length-preserving, and unauthenticated; it is excluded from the security claims.

III. Related Work

a) Anonymity and censorship resistance: Tor provides layered encryption and onion routing through relays [1]. Pluggable transports such as obfs4 and ScrambleSuit modify the observable protocol shape [2], [3], and format-transforming encryption maps ciphertext to a target format [4]. These systems target a different problem: hiding the fact that a flow is Tor or making it resemble another protocol. TorShield does not attempt either. Work on traffic morphing changes packet sizes and timing [5], and measurement studies show that naive obfuscation can be fingerprinted [6]. Those results motivate our explicit non-goal: padding to 64-byte buckets reduces exact-length leakage but does not make the traffic unobservable.

b) Network-layer protection: IPsec provides authenticated and encrypted IP traffic with a standards-based policy and key-exchange framework [7]. WireGuard is a compact kernel UDP tunnel with a Noise-based handshake, cryptokey routing, and forward secrecy [8]. OpenVPN is a userspace TLS-based tunnel. TorShield shares the UDP/AEAD idea with WireGuard but differs in deployment and scope: it exposes transparent Netfilter steering and a TC/eBPF steering path, uses manually configured per-peer keys, and has no handshake. We do not claim better security or performance than WireGuard; for production privacy, WireGuard or IPsec is the appropriate choice.

c) Programmable packet processing: eBPF and TC allow packet classification and redirection in the kernel [9]. TorShield uses a clsact egress classifier only for steering; the AEAD remains in the device's `ndo_start_xmit` because the BPF helper set cannot call the synchronous in-kernel ChaCha20/Poly1305 path. The fail-closed guard is a separate Netfilter hook, which creates the ordering problem analyzed in Section IV-F.

IV. System Design

A. Overview and Modes

TorShield is a loadable kernel module. It is inert until a key and at least one peer are configured, so loading it does not disrupt a host. Fig. 1 shows the shared datapath, and Table I lists the four modes. The default is `tunnel`. `hybrid` and `tc` add alternative egress-steering mechanisms but keep the same wire format and the same Netfilter ingress decapsulation. `stream` is the legacy length-preserving transform and is not recommended.

B. Wire Format and AEAD

An outer packet is

```
outer IP | UDP:4789 | 16-byte header | padded
          ciphertext | 16-byte tag.
```

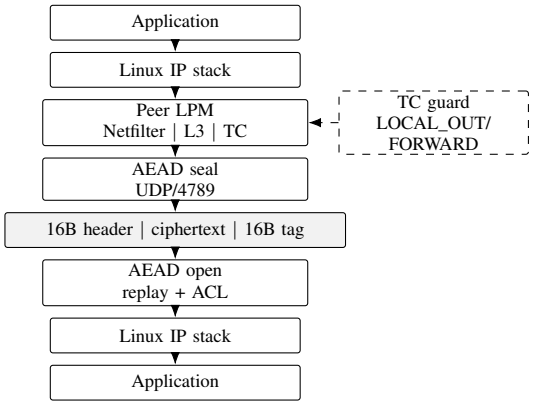


Fig. 1. TorShield v4 datapath. All egress modes share one AEAD wire format; the TC guard drops peer-destined packets when the classifier or qdisc is missing.

TABLE I
Datapath modes. Tunnel, hybrid, and TC share one wire format.

Mode	Egress behaviour
`tunnel`	Netfilter `POST_ROUTING` matches the inner destination against peer prefixes, encapsulates, and returns `NF_STOLEN`. Default.
`hybrid`	A route into a `torshield` L3 netdevice is encapsulated by `ndo_start_xmit`; other peer traffic still uses Netfilter.
`tc`	A clsact/eBPF classifier on the underlay redirects matching inner packets to a `torshield` device; a Netfilter guard at `LOCAL_OUT`/`FORWARD` drops peer packets if the classifier is missing.
`stream`	Legacy length-preserving payload transform; no authentication. Excluded from security claims.

The 16-byte header is the AEAD associated data. It contains a two-byte `TS` magic, a one-byte version, a one-byte flags field (bit 0 indicates an inner IPv6 packet), a 32-bit key identifier (zero selects the legacy global PSK), and a 64-bit big-endian per-peer sequence number. The nonce is the 4-byte XOR fold of the outer source and destination addresses followed by the 8-byte sequence number. The plaintext is the complete inner IP packet, zero-padded to a multiple of 64 bytes; the inner header carries the true length and is parsed only after authentication. The tag is 16 bytes. ChaCha20–Poly1305 follows RFC 8439 [10]. The ChaCha20 block function is taken from the in-kernel helper and Poly1305 is implemented in the module; the path is synchronous because the asynchronous crypto API cannot be used from a softirq Netfilter hook.

The outer IPv4 identification field is assigned from a per-packet counter. This matters when a large inner UDP datagram is split into multiple outer IP fragments: a constant zero would make fragments from different datagrams indistinguishable to reassembly. The outer UDP checksum is zero over IPv4, which is legal; integrity is provided by the AEAD tag.

a) Port choice: The default outer port is 4789, the IANA VXLAN port. Wireshark and similar tools may heuristically label the outer datagrams as VXLAN. That is a display heuristic: the payload is the `TS` header and AEAD ciphertext, not a VXLAN encapsulation. The port can be changed at

module load.

C. Peer Table and Cryptokey Routing

Each peer entry contains a 32-byte key, a deterministic 32-bit key identifier (full FNV-1a of the key, with zero reserved), an outer endpoint, and up to eight allowed inner prefixes. Egress selects the peer by longest-prefix match on the inner destination. Ingress looks up the peer by the wire key identifier (or by outer source for a legacy global-PSK peer), authenticates, and then checks the decrypted inner source against that peer's allowed prefixes. A packet whose inner source is outside the allowed set is dropped and counted. The key identifier is a selector, not an authenticator: an observer can read it, but cannot produce a valid tag without the key. After a packet authenticates, a peer configured with a per-peer key may roam to the new outer source address, so a mobile endpoint can change addresses without a control message.

D. Netfilter Datapath

The module registers hooks per network namespace with `register_pernet_subsys`. In tunnel mode, egress runs at `NF_INET_POST_ROUTING` with priority `NF_IP_PRI_LAST`. It skips outer packets, link-local and loopback sources, and packets already marked by the tunnel. It looks up the peer by inner destination; if there is no peer, the packet continues unchanged. Otherwise it handles GSO by segmenting the super-skb into `gso_size` chunks, assigns each segment its own sequence number, and encapsulates each one. The original skb and each segment are released after they are copied. The hook returns `NF_STOLEN`. On ingress, `NF_INET_LOCAL_IN` runs after IP reassembly. It accepts only UDP packets to the configured port; a packet with a non-`TS` header is left alone. A framed packet is rate-limited (if enabled), linearized, authenticated, replay-checked after authentication, ACL-checked, and re-injected with `netif_rx`. Any parsing, authentication, replay, ACL, or allocation failure in tunnel mode drops the packet and increments a counter; it is never forwarded as plaintext.

A TCP SYN is clamped to an MSS of 1360 bytes for IPv4 and 1340 bytes for IPv6 so that the outer packet fits in a 1500-byte MTU. Large inner UDP datagrams are allowed to fragment at the outer IP layer; the receiver reassembles and then linearizes before the AEAD. The current implementation verifies 4 KB and 60 KB inner datagrams.

E. Hybrid L3 Device

`torshield0` is an `ARPHRD_NONE` L3 netdevice created with `ip link add ... type torshield`. It has no L2 header, no ARP, no queueing discipline, an MTU of 1360, and no GSO/TSO offloads. A route such as `ip route add 10.0.0.2/32 dev torshield0` sends selected prefixes into `ndo_start_xmit`, which reuses the same encapsulation functions. The outer route lookup is pinned to the underlay interface supplied as `IFLA_LINK`; this prevents the outer route from matching the route into the device and recursing. In hybrid mode the Netfilter egress hook skips packets whose route already points at a `torshield` device, while all other peer traffic keeps the transparent path. A TC redirect hands the device a full L2 frame, so `ndo_start_xmit` strips the link header before parsing the inner IP packet.

F. TC Steering and the Fail-Closed Guard

TC mode replaces routes with a clsact egress classifier on the underlay. The BPF program `ts_tc_egress` looks up the inner destination in LPM tries for IPv4 and IPv6, skips multicast and link-local traffic, and redirects a match to the `torshield` device ifindex stored in its `cfg` map. Outer frames carry mark `0x5454` so they are not redirected twice. If a peer matches but `cfg[0]` has no redirect target, the program returns `TC_ACT_SHOT` rather than `TC_ACT_OK`; a matching packet must never fall through to the wire.

The difficult case is classifier removal. Netfilter `POST_ROUTING` runs before TC egress. In TC mode the Netfilter egress hook is bypassed (otherwise packets would be encapsulated twice), so relying on the classifier alone would be fail-open if the qdisc or program disappeared. TORSHIELD therefore registers IPv4/IPv6 `NF_INET_LOCAL_OUT` and `NF_INET_FORWARD` hooks at the first priority, both before `POST_ROUTING`. For a peer-destined packet, the guard uses `nf_hook_state.out` (not `skb_dst`, which can still describe the input route at `FORWARD`) and accepts only if:

1) the output device is a `torshield` device, which encapsulates in `ndo_start_xmit`; or
2) the output device still has a live TC egress classifier—a TCX program named `ts_tc_egress` in `dev->tcx_egress`, or a live `cls_bpf` filter in the clsact mini-Qdisc (the style the loader uses today).

Otherwise the guard returns `NF_DROP` and increments `tc_guard`. A local `sendto()` then fails with `EPERM`; no clear frame reaches the underlay. The TCX branch is compile-tested only; the runtime tests use the clsact path.

The guard proves only that a classifier is attached; it does not prove that the classifier's maps are complete. `torshield-tc attach` reads every `allowed=` prefix from `/proc/torshield/peers` and fills the LPM tries before switching the mode to `tc`. If a peer is added, deleted, or cleared while the mode is `tc`, the kernel forces the mode back to `tunnel` until the loader reruns. Direct map edits or a hand-loaded classifier are outside the guarantee. The loader and the peer-list update are not one atomic transaction, so operators must not reconfigure peers concurrently with `attach`.

a) Residual race: The guard and the later TC egress hook read the attachment separately. If a classifier is deleted after the guard accepts a packet but before that packet reaches egress, that one in-flight packet can still leave in clear. The deletion tests in Section VI prove fail-closed behaviour for all subsequent packets, not a zero-race guarantee. Closing this requires a last-chance `ndo_start_xmit` check or a single authoritative steering point; neither is implemented.

G. Replay, Rate Limiting, and Manual Rekey

The receiver keeps a 1024-packet sliding window per peer. A sequence of zero is invalid. The window is advanced only after successful authentication, so an off-path packet cannot consume a window slot. The window was widened from 64 packets after four-stream native traffic produced 229 false replay drops; with 1024 packets the same run produced zero.

An optional per-source fixed-window limiter (`ts_rl_pps`, default zero) runs before `skb_linearize` and Poly1305. It bounds the crypto work a single source can force, but it is not a cookie challenge and the source address is spoofable. It is disabled by default so that measurements are not capped.

Per-peer keys can be rotated manually. `rekey-peer` moves the current key to a previous slot and starts sending with the new key; the receiver accepts both key identifiers until `rekey-commit` erases the previous key. A second rotation before commit is rejected, so the oldest key cannot be silently discarded. This protects packets already in flight, but both ends must be reconfigured and traffic can drop between the two commands. There is no handshake and no forward secrecy.

V. Implementation

The canonical module is `module/LKM/torshield.c`; the TC classifier and loader are in `module/LKM/tc/`. The control plane is `/proc/torshield/{psk,peers,mode,stats}` with a thin `torshieldctl` wrapper. The peer file accepts `+endpoint key HEX allowed PREFIX,...` and `-endpoint` / `clear`; the stats file exposes tunnel, device, ACL, roaming, rate-limit, and guard counters. The module registers ten hooks per network namespace: six datapath hooks (IPv4/IPv6 `PRE_ROUTING`, `POST_ROUTING`, `LOCAL_IN`) and four guard hooks (IPv4/IPv6 `LOCAL_OUT`, `FORWARD`).

The TC loader is a libbpf program. It loads `torshield_tc.o`, fills the LPM and `cfg` maps, attaches the program to the underlay's clsact egress, and only then writes `tc` to the mode file. `detach` writes `tunnel` first and then removes the qdisc. A watchdog can poll the attachment and revert the mode, but it is a recovery aid, not the security boundary.

The module builds cleanly with `make W=1` against Ubuntu 24.04 kernels 6.8.0-147 and 6.17.0-42, and against the native 6.18.54.1 headers used for the performance runs. The 6.16 ChaCha API change (opaque state array to `struct chacha_state`) is handled with a version-conditional shim. The TC object and loader build with `-Wall -Wextra` clean.

VI. Evaluation

A. Methodology and Environment

Correctness, deletion, and interoperability tests run in an Ubuntu 24.04 QEMU 8.2 guest with kernel 6.8.0-147 and TCG (no KVM). The guest uses network namespaces `n1`/`n2` connected by a veth pair; a third namespace `n3` is used for the forwarded-path test. Throughput is measured separately on the native WSL2 kernel 6.18.54.1 (Hyper-V, 16 vCPUs), because TCG CPU emulation is not a throughput measurement. The native host is a VM, not bare metal. The direct-veth baseline is about 100 Gbit/s, so the link is not the bottleneck; the tunnel is CPU-bound.

TABLE II
Verified correctness and interoperability results. All values are from captured runs, not projections.

Test	Result
Tunnel, IPv4 TCP 1 MB	1 048 576/1 048 576 bytes (100%)
Tunnel, IPv6 TCP 256 KB	262 144/262 144 bytes (100%)
Independent AEAD, tunnel capture	1054/1054 frames accepted
Tunnel, 4 KB inner UDP	4000/4000 bytes; 3 outer fragments
Tunnel, 60 KB inner UDP	60000/60000 bytes; 41 outer fragments
Replay / tamper / unknown peer	each dropped; counters exact
Hybrid, IPv4 TCP 1 MB	100%; 807/807 frames accepted
Hybrid, IPv6 TCP 256 KB	100%; 209/209 frames accepted
Cryptokey routing, v4/v6 echo	256/256 and 64/64
TC, IPv4 UDP echo	256/256; `dev_out` 512; `tun_out` 0
TC, IPv6 UDP echo	64/64; `dev_out` 647 (cumulative); `tun_out` 0
TC detach fallback	mode `tunnel`; echo 16/16

Each run starts from a clean module, configures keys and peers, and captures outer packets. A host-side Python verifier independently reconstructs the nonce and AAD and decrypts each frame with the `cryptography` package. Counters are read from `/proc/torshield/stats`. The test harness and per-stage scripts are part of the artifact.

B. Correctness and Interoperability

Table II summarizes the functional results. The IPv4 TCP transfer was byte-exact, the IPv6 transfer was byte-exact, and the independent verifier accepted every captured frame. The large-UDP cases exercise outer fragmentation and reassembly. Replay, tamper, and unknown-source packets were dropped with exact counter increments. TC and hybrid paths used distinct inner and outer prefixes, so a successful transfer also demonstrates cryptokey routing rather than address coincidence.

C. Wire-Format Verification

Independent AEAD verification accepted 1054/1054 tunnel frames, 807/807 hybrid IPv4 frames, 209/209 hybrid IPv6 frames, 210/210 TC IPv4 frames, and 68/68 TC IPv6 frames with the correct key. The TC captures accepted 0/210 and 0/68 when verified with the global PSK, confirming that key selection is actually in use. The native IPv6 pcap capture was empty in the WSL2 harness, so the IPv6 wire claim rests on the QEMU 68/68 capture; native IPv6 was verified functionally by echo and counters.

D. TC Fail-Closed Behaviour

Table III reports the deletion tests. In the `LOCAL_OUT` test, a local socket sends eight packets while the classifier is deleted; all eight fail with `EPERM`, the peer server receives zero, `tc_guard` increases by eight, and both `dev_out` and `tun_out` remain unchanged. Deleting the clsact qdisc

TABLE III
TC FAIL-CLOSED TESTS. VALUES ARE DELTAS OVER THE TEST WINDOW.

Condition	Observed result
`LOCAL_OUT`, classifier deleted	0 sent / 8 blocked (`EPERM`); server 0; `tc_guard` +8; `dev_out` +0; `tun_out` +0
`LOCAL_OUT`, qdisc deleted	same: 0/8; server 0; `tc_guard` +8; `dev_out` +0; `tun_out` +0
`LOCAL_OUT`, classifier restored	echo 16/16; detach fallback 8/8
`FORWARD`, no classifier	`n2` ingress 8; `n3` outer 0; `tc_guard` +8; `dev_out` +0
`FORWARD`, classifier present	`tc_guard` +0; `dev_out` +8; `n3` `tun_rx` +8
`FORWARD`, classifier deleted	`n3` outer 0; `tc_guard` +8; `dev_out` +0
Peer add/del/clear in `tc`	each forces mode `tunnel`
Loader auto-fill, no `-4/-6`	2 IPv4 + 2 IPv6 prefixes; echo 16/16

produces the same result. Reattaching the classifier restores a 16/16 echo; detaching falls back to the Netfilter path at 8/8.

The `FORWARD` test injects raw frames on `n1`, so the sender's `LOCAL_OUT` hook is bypassed. With no classifier, `n2` receives eight forwarded inner packets, `n3` sees zero outer packets, `tc_guard` increases by eight, and `dev_out` does not move. With the classifier present, `tc_guard` stays flat, `dev_out` increases by eight, and `n3` records eight `tun_rx`. Deleting the classifier returns to zero outer packets and a guard increase of eight. This is the empirical basis for the fail-closed claim, with the residual race noted in Section IV-F.

E. Manual Rekey and Rate Limiting

On the native kernel, a rekey changed the egress key identifier from 4290739141 to 1666914373. During the grace period, four payloads using the old and new keys were all accepted. After `rekey-commit`, one old-key packet was rejected as non-peer traffic: `tun_in` increased by four, `tun_peer` by one, and `tun_replay` stayed zero. A same-key no-op returned success; a second rotation before commit, a commit without a previous key, and invalid hex all returned errors without changing state.

With `ts_rl_pps=50`, 950 of 1000 synthetic packets were rejected before the AEAD and 50 reached it; a legitimate 64-packet echo still completed 64/64. This demonstrates the limiter's placement, not a general DoS defense.

F. Performance

Table IV reports the native VM measurements. The baseline is direct veth with four TCP streams. The tunnel is CPU-bound: one stream reaches about 0.91–0.97 Gbit/s and four streams reach 3.02–3.35 Gbit/s. A separate 30 s four-stream run sustained 3.40 Gbit/s with zero tunnel drops and zero replays; `SUnreclaim` stayed flat at 63–68 MB. A 100-ping RTT run averaged 0.052 ms through the tunnel; this is a VM measurement, not a tail-latency study, and we do not report the draft's 0.5 ms per-packet figure.

TABLE IV
NATIVE WSL2/HYPER-V MEASUREMENTS (KERNEL 6.18.54.1, 16 VCPUS). NOT BARE METAL.

Test	Result
Baseline, direct veth, 4 streams	100.98 / 99.76 / 97.84 Gbit/s
Tunnel, 1 stream	909.7 / 921.5 / 965.4 Mbit/s
Tunnel, 4 streams	3345.6 / 3018.1 / 3203.1 Mbit/s
Tunnel, 30 s sustained, 4 streams	3.40 Gbit/s; `tun_drop` 0; `tun_replay` 0
Unreclaimable slab, 30 s run	flat at 63–68 MB
Tunnel RTT, 100 pings	0.052 ms average

TABLE V
DEFECTS EXPOSED BY THE EVALUATION AND FIXED IN THE CURRENT CODE.

Defect	Evidence and fix
GSO skb leak	30 s four-stream run drove `SUnreclaim` to about 7.4 GB and panicked; `consume_skb(seg)` after encapsulation; same run flat at 63–68 MB.
Replay window too small	64-packet window produced 229 false replay drops and 107 retransmits on four streams; 1024-packet window produced zero replays.
Fragmented-skb read	Reassembled outer datagrams failed AEAD because the ciphertext was read only from the linear head; `skb_linearize()` before the AEAD.
Outer IPv4 ID zero	All outer datagrams carried ID 0; per-packet counter and IPv4 checksum fix.

G. Defects Found and Fixed

The evaluation exposed four defects that static review did not. Table V lists them. The GSO leak is the most serious: the egress loop copied each segment into an outer skb but did not release the original segment, so a 30 s four-stream run drove unreclaimable slab to about 7.4 GB and panicked the VM. The replay window was too small for parallel-queue reordering. The receive path read a reassembled fragmented skb through a linear pointer. The outer IPv4 ID was always zero. All four are fixed in the current code and covered by the regression runs.

H. Withdrawn Claims

The initial draft reported 1% additional CPU, 0.5 ms per packet, a 50 KB kernel footprint, 5% throughput loss, 72-hour stability, and a matrix of Ubuntu, CentOS, and Debian kernels. Those numbers are not reproduced by the current system and are not used here. In particular, we did not measure CPU utilization or per-packet processing time; the module-size and long-duration stability claims are not part of this study; and the build matrix in this paper is Ubuntu 24.04 with kernels 6.8.0-147 and 6.17.0-42 plus the native 6.18.54.1 VM.

I. Threats to Validity

The throughput numbers come from a WSL2/Hyper-V VM, not bare metal, and from veth rather than a physical NIC. Checksum and offload behavior on a real NIC is not validated. The TC deletion tests are IPv4 only; the IPv6 guard hooks build but are not separately tested for removal. The TCX branch is compile-tested only. The rekey test is IPv4 only.

The native IPv6 pcap capture is empty in this harness, so the IPv6 wire evidence is the QEMU capture. The performance suite does not isolate CPU time per packet, and the RTT run is short. These limitations bound the claims; they do not affect the byte-exact correctness or the deletion counters.

VII. Security Analysis

a) Protections: The AEAD provides confidentiality and integrity for the inner IP packet, including its addresses and transport headers. The 64-bit per-peer sequence number and the 1024-packet window reject replays after authentication. The key identifier selects a per-peer key; the inner source ACL prevents a peer from injecting a source prefix it does not own. Endpoint roaming occurs only after authentication, so an off-path attacker cannot redirect a peer by spoofing an outer source. In tunnel and hybrid modes, all authentication and parsing failures drop the packet. In TC mode, the guard drops peer-destined packets when the classifier is missing, and the classifier itself drops a matching packet when its redirect target is unset. Padding to 64-byte buckets reduces exact-length leakage.

b) Residual risks: Outer addresses, ports, the `TS` magic, key identifier, sequence number, ciphertext length, direction, timing, and approximate size bucket are visible. The system does not resist traffic analysis or DPI. There is no handshake, no forward secrecy, and no automatic key agreement; a compromised key exposes the traffic it protects, and a compromised endpoint exposes plaintext. The peer table, key, and mode are global across network namespaces. The rate limiter is a fixed-window filter, not a cookie challenge, and the source is spoofable. The TC guard has a narrow check-to-egress race; the deletion tests prove fail-closed behavior for subsequent packets, not absolute zero-race closure. The TCX path is not runtime-tested. TC and hybrid mode are prototypes, not production-hardened transports. The legacy stream mode has no integrity and is excluded.

c) Not Tor: TorShield does not provide anonymity. It does not use onion routing, cover traffic, or a distributed relay network. Its name is historical. A user who needs anonymity should use Tor; a user who needs a production VPN should use WireGuard or IPsec.

VIII. Discussion and Lessons

Three lessons recur. First, transparency is not free: the same packet can be selected by Netfilter, a route, or TC, and the ordering of those mechanisms determines both correctness and failure behavior. The TC guard exists because `POST_ROUTING` runs before TC egress; a fail-closed design needs a check before the point where the packet can escape. Second, GSO and skb ownership are the main kernel pitfalls. The byte-exact tests passed while the GSO path leaked memory, and only a sustained run exposed it. Third, replay-window sizing is a property of the deployment's parallelism, not a constant: a 64-packet window was adequate for a single stream and wrong for four streams on parallel queues.

The negative result about the XOR prototype is also a lesson. A length-preserving transform can be described as "obfuscation" and still be trivially invertible; without authentication and a nonce discipline, it is not a security mechanism. Replacing it with an AEAD forced a different framing, a sequence number, a replay window, and a key identifier—and made the evaluation meaningful.

IX. Conclusion and Future Work

TorShield is a transparent in-kernel authenticated tunnel with three steering paths and a kernel-side fail-closed guard for TC. The current code is byte-exact on IPv4 and IPv6 TCP, survives independent AEAD verification, drops replay/tamper/unknown-source packets, and blocks subsequent local and forwarded packets when the TC classifier is removed. On a native VM it sustains 3.40 Gbit/s over four streams for 30 s with flat unreclaimable memory. The paper also reports the defects that the evaluation found and the claims that the earlier XOR draft cannot support.

Future work is concrete: close the guard-to-egress race with a last-chance transmit check or a single steering point; add a handshake and forward secrecy; move the control plane to netlink and make state per-network-namespace; deliver decapsulated packets through the tunnel device so policy routing and `rp_filter` see them; advertise GSO/TSO on the device and segment in the driver; run the deletion suite for IPv6 and the TCX path; and repeat the throughput study on bare metal with a physical NIC. Until then, TorShield remains a research and teaching artifact, not a production replacement for WireGuard or IPsec. The code and test harness are available at https://github.com/roshansrikanth21/TorShield.

References

[1] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proceedings of the 13th USENIX Security Symposium*, 2004, pp. 303–320.

[2] The Tor Project, "obfs4: A pluggable transport," https://gitlab.com/yawning/obfs4, 2024, accessed 2026-10-22.

[3] P. Winter, T. Pulls, and J. Fuss, "Scramblesuit: A polymorphic network protocol to circumvent censorship," in *Proceedings of the 12th ACM Workshop on Privacy in the Electronic Society*, 2013, pp. 213–224.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Protocol misidentification made easy with format-transforming encryption," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, 2013, pp. 61–72.

[5] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[6] A. Houmansadr, C. Brubaker, and V. Shmatikov, "The parrot is dead: Observing unobservable network communications," in *Proceedings of the 2013 IEEE Symposium on Security and Privacy*, 2013, pp. 65–79.

[7] S. Kent and K. Seo, "Security architecture for the internet protocol," RFC 4301, IETF, 2005.

[8] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[9] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path: Fast programmable packet processing in the operating system kernel," in *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2018, pp. 54–66.

[10] Y. Nir and A. Langley, "Chacha20 and poly1305 for ietf protocols," RFC 8439, IETF, 2018.